

RFID Go GPS

Technická příručka pro vývojáře

Verze aplikace 3.153

Architektura, třídy, datové toky a implementační detaily kódu. Určeno pro vývoj a údržbu
- ne pro terénní provoz.

Tato příručka je třetí dokument v sadě – určená výhradně pro vývoj a údržbu kódu. Na rozdíl od `prirucka-teren.md` (terén) a `prirucka-uzivatele.md` (uživatel) popisuje architekturu, třídy, datové toky a implementační detaily bez návodu pro operátora.

1. Architektura aplikace

1.1 Jednoaktivitní model

Aplikace má jednu aktivitu (`MainActivity`) a žádné fragmenty. Veškerý provozní stav (UDU, výhybka, čip, workflow, GPS, CSV) žije v instančních polích `MainActivity` (~5 600 řádků). Logika je rozdělena do samostatných balíčků podle domény; UI orchestrace zůstává v aktivitě.

```

`  app/src/main/java/com/rfidw/app/ └─ ui/ | └─ MainActivity.java ←
orchestrátor (workflow, GPS, UI) | └─ CsvAdapter.java ← RecyclerView pro
tabulku CSV └─ epc/EpcModel.java ← sestavení/rozkład 24hex EPC └─ data/
| └─ Tudu.java ← doménový model TUDU/výhybka/R0 větev | └─
DzsDatabase.java ← SQLite DZS, indexace okolí GPS | └─ DzsIndexCache.java
← disková cache .pidx (verze 21) | └─ VyhybkaGpsStore.java ← kompaktní
pole GPS bodů výhybek └─ csv/ | └─ CsvStore.java ← in-memory tabulka +
index čipů | └─ CsvStorage.java ← cesta Download/, MediaStore | └─
CsvRecordBuilder.java ← factory řádku CSV └─ rfid/UhfManager.java ← obal
Chainway RFIDWithUHFUART └─ location/LocationCache.java └─
kmext/KmExtResolver.java ← KM_EXT z OD/D0/KM_REF `

```

1.2 Vlákna a exekutory

Executor	Účel
io	RFID zápisy, persist CSV, hash DB na pozadí
gpsIo	Otevření DB, proximity indexace, GPS lookup
ui Handler	Veškeré aktualizace UI (<code>runOnUiThread</code>)

Pravidlo: RFID a SQLite nikdy neblokují hlavní vlákno. `UhfManager.init()` je výjimka – volá se synchronně z `onCreate`.

1.3 Perzistence mezi spuštěními

SharedPreferences rfidgogps:

Klíč	Typ	Význam
idRfid	long	další pořadové číslo tagu (min. 400)
epcTemplateMode	boolean	ON = šablona EPC, OFF = TID→EPC
tuduModeGps	boolean	true = GPS auto-výběr, false = ruční
gpsTestMode	boolean	simulace GPS z DB
testLat / testLon	float	souřadnice testovacího režimu
dbSourcePath	String	cesta k poslední DB
dbDisplayName	String	zobrazovaný název DB
dbSourceUri	String	content:// URI (SAF picker)

Legacy prefs rfidgo - záloha idRfid pro migraci ze starší aplikace.

Soubory mimo prefs: Download/rfid_go_gps_output.csv, files/dzs/ (kopie DB + .pidx cache), hash sidecary hash2_*.txt.

1.4 Závislostní graf

```
`` MainActivity ┤─ UhfManager ────────────> RSCJA DeviceAPI (nativní
               │ ┤─ LocationCache ──────────> FusedLocation / GPS provider ┤─
DzsDatabase | ┤─ DzsIndexCache (.pidx) | ┤─ VyhybkaGpsStore | ┤─ Tudu
(model)    | ┤─ KmExtResolver ┤─ CsvStore ──> CsvStorage (MediaStore /
soubor)    | ┤─ CsvRecordBuilder ┤─ EpcModel (čistá logika, testovatelná na
JVM)      ``
```

2. Provozní workflow (MainActivity)

2.1 Tři hlavní kroky UI

Indikátor nahoře (karty 1–3):

Krok	Flag	Podmínka splnění
1 UDU	step1Done	TUDU + výhybka vybrány, nebo vyplněný formulář hranice TUDU
2 Načtení	step2Done	Zvolen preset výkonu (activePowerPresetInKoleji != null)
3 Hotovo	step3Done	Úspěšné zamčení tagu v řetězci

Barvy stavu: COLOR_STATUS_READY (zelená), BUSY (šedá), ERROR (červená), WARNING / GPS_WAIT (oranžová).

2.2 Pod-workflow zápisu (4 kroky)

Pole wfStepStates[4] + konstanty:

```
`` WF_STEP_EPC = 0 přepis EPC WF_STEP_CSV = 1 zápis řádku CSV WF_STEP_PWD = 2 access heslo WF_STEP_LOCK = 3 zamčení ``
```

Stavy každého kroku: PENDING → ACTIVE → OK / FAIL.

Řetězec (chainWorkflow = true): spouště spustí všechny kroky postupně. Po úspěšném LOCK → dialog „Načetli jste“ (scanDoneAwaitingConfirm). Druhý stisk spouště = onScanDoneContinue() (posun čipu, idRfid++).

Nechain režim: tlačítka v panelu Pokročilé volají doWrite(), doWritePassword(), doLock() samostatně.

2.3 Spouště čtečky

```
``java private static final int[] TRIGGER_KEYS = { 139, 280, 293, 311, 312, 522, 523, 0x3E8 }; ``
```

onKeyDown() → podle kontextu:

- kontrolaActive → runKontrolaRead()
- scanDoneAwaitingConfirm → onScanDoneContinue()
- jinak (bez delete dialogu) → runTriggerAction()

runTriggerAction() validuje: výkon, výběr větve u 3častové výhybky, validitu EPC (v template módu), formát NEW PWD → refreshGpsAtWorkflowStart() → chainWorkflow = true → doWrite().

2.4 EPC režimy

epcTemplateMode	Metoda UHF	EPC obsah
false (výchozí)	writeEpcFromTid()	normalizovaný TID (24 hex)
true	writeEpc()	EpcModel.buildEpc() ze šablony

Po zápisu: `onWriteDone()` → pokud `cbAutoCsv` → `saveRowToCsv()` → `doWritePassword()` → `onPwdWriteDone()` → `doLock()` → `onLockDone()`.

2.5 Post-cyklus: posun stavu

`onTagCycleComplete()`:

- `epc.idRfid++` (persist do prefs)
- `advanceCastAndVyhybka()` – další chybějící čip nebo další výhybka v UDU
- `firstMissingCast()` – hledá mezery v CSV pro aktuální TUDU/výhybku
- U 3částové výhybky: čipy 1–3 mapují na větve JAZYK/HLAVNI/VEDLEJSI nezávisle na čísle čipu

2.6 GPS auto-výběr

```
`` LocationCache.getSnapshot() | ▼ scheduleGpsTuduLookup() [throttle: min 5  
m pohyb, 1 s interval] | ▼ DzsDatabase.ensureProximityLoaded(lat, lon) | ▼  
findNearestDistinctFullTudu() / findNearest() | ▼ applyGpsMatch(GpsMatch)  
→ currentTudu, currentVyhybka, epc.* ``
```

Zámky: ruční výběr TUDU nastaví `gpsTuduLocked`; ruční výhybka `gpsVyhybkaLocked`. Po dokončení všech čipů na výhybce se zámky uvolní.

Režim Ručně (`tuduModeGps = false`): GPS lookup se neprovádí; operátor vybírá z dialogu `showTuduPicker()`.

Testovací GPS (`gpsTestMode`): `LocationCache.setTestOverride()` – do CSV jde prefix TEST v GPS DATE.

2.7 Režim Kontrola

- Overlay `kontrolaOverlay` přes celou obrazovku
- Vlastní preset výkonu (oddělená `RadioGroup`)
- `runKontrolaRead()` → `UhfManager.readSingle()` → `CsvStore.findAllRowsByTag(epc, tid)`
- Více shod → navigace `kontrolaMatchIndex` šipkami
- Žádný zápis do tagu ani CSV

2.8 Hranice TUDU (čip 5)

Konstanta `CAST_TUDU_BOUNDARY` = 5.

Aktivace: tlačítko Hranice TUDU → `showTuduBoundaryForm()` dialog.

Stav:

- `tuduBoundaryMode` = true
- `epc.cast` = 5
- GPS auto-výběr vypnut
- UI: „objekt“ místo „výhybka“ (`tuduBoundaryObjektLabels`)
- CSV: prázdná `POLOHA/RO_ID`, ruční `tuduBoundaryVyhybkaLabel`, `tuduBoundaryKmExt`

Po zápisu: `finishTuduBoundaryWriteCycle()` → exit boundary mode → force GPS re-lookup.

Obnova z CSV: `restoreTuduBoundaryFromRow()` když `cast == 5`.

3. EpcModel - kódování EPC

Soubor: `epc/EpcModel.java`. Čistá Java logika - testovatelná mimo Android (JVM harness).

3.1 Layout 24 hex znaků

EPC = 6 Gen2 wordů, bank EPC, ptr 2, len 6:

Segment	Znaky	Pole	Kódování	Příklad
rok	4	year	literál, padRight '0'	2026
TUDU 1-4	4	tudu[0..3]	uppercase	1501
TUDU 5	2	tudu[4]	ASCII hex	J → 4A
TUDU 6	2	tudu[5]	číslice → %02d, jinak ASCII hex	1 → 01
výhybka	3	vyhybka	dekadicky %03d mod 1000	10 → 010
část	1	cast	hex mod 16	1 → 1, 10 → A
ID_RFID	8	idRfid	dekadicky %08d mod 10 ⁸	30001 → 00030001

Součet: 4+4+2+2+3+1+8 = 24.

Příklad: TUDU 1501J1, výhybka 10, čip 1, ID 30001: `` 2026 + 1501 + 4A + 01 + 010 + 1 + 00030001 = 202615014A01010100030001 ``

3.2 API

Metoda	Popis
<code>f1Year() ... f7IdRfid()</code>	jednotlivé segmenty jako hex string
<code>buildEpc()</code>	spojení 24 znaků
<code>buildEpcPreview()</code>	formát xxxx-xxxx-...
<code>isValid()</code>	délka 24 + hex kontrola
<code>decode(String epc24)</code>	statický rozklad → Decoded

3.3 Decode - edge cases

- TUDU 6. znak: 01-09 → dekadická číslice; jinak ASCII z hex
- 00 na pozici TUDU 6 = prázdný 6. znak
- cast se čte jako hex integer (ne dekadicky)

4. Doménový model Tudu

Soubor: data/Tudu.java.

4.1 Hierarchie

```
` Tudu (code: "1501J1") └─ Vyhybka (cislo: 10, iob: "A") └─ castMin /  
castMax (1–3 nebo 1–4) └─ List<RoBranch> └─ roId, poloha, kmExtChip1,  
kmExt0ther ``
```

UDU = prvních 5 znaků TUDU (uduCode()). Do EPC a CSV jde plný 6znakový TUDU.

4.2 Typy výhybek (POLOHA)

První znak POLOHA	Typ	castMax
J	3částová	3
C	4částová	4

Větve (2. znak POLOHA):

2. znak	Větev
A, C	Hlavní (isHlavniPoloha)
B, D	Vedlejší (isVedlejsiPoloha)

4částové sady:

Sada	Pár 1 (čipy 1-2)	Pár 2 (čipy 3-4)
ABCD	CA	CB
EFGH	CG	CH

Metody: isCastPair1Poloha, isCastPair2Poloha, fourPartFamily(),
castFourPartLabel(cast).

4.3 IOB písmeno

Vyhybka.iob – volitelné písmeno z DB (např. 10A). displayLabel() → "10" nebo "10A".
Normalizace: normalizeIob() bere první alfanumerický znak, ignoruje "null".

4.4 KM_EXT na větvi

RoBranch.kmExtChip1 – hodnota pro čip 1 (odpovídá KM_REF z DB).
RoBranch.kmExt0ther – druhý konec OD/DO pro čipy 2+.

Odvození: KmExtResolver.fromOdDoKmRef(od, doVal, kmRef).

5. DZS databáze a indexace

Implementace: DzsDatabase.java, DzsIndexCache.java, VyhybkaGpsStore.java.
Doplňkový dokument (částečně zastaralý u plné indexace): docs/INDEXACE_DZS.md.

5.1 SQLite tabulky

Tabulka	Účel
DZS_SUPER_RO_TPI	výhybky, TUDU, POLOHA, RO_ID, OD/DO
DZS_SUPERTRA_GPS_KM	GPS body tratě, KM_EXT

Dynamické sloupce: PRAGMA table_info → třídy GpsColumns, RoColumns mapují aliasy (LAT/LATITUDE, COBJEKT/VYHYBKA, ...).

Filtry při indexaci:

- POLOHA prázdná nebo NULL → vyřazeno
- RO_ID operátor TUDC (EXCLUDED_RO_OPERATOR) → vyřazeno

5.2 Proximity index (aktuální model od v3.141)

Nepoužívá se plná indexace celé DB. Při otevření:

- Bbox $\pm 0,04^\circ$ (~4 km) kolem GPS polohy
- SQL JOIN GPS + RO pro body v bboxu
- Výsledek: roByPairKey mapa + VyhybkaGpsStore
- Lazy SpatialGrid pro nearest-neighbor

Reload: při posunu > 3 km (PROXIMITY_RELOAD_MOVE_KM) se okolí doindexuje znovu.

5.3 SpatialGrid

- Buňka CELL_DEG = 0.005 (~550 m)
- MAX_RING = 40 prstenců
- Hledání: expandující čtvercové prstence od středu
- Vzdálenost: degree space s $\cos(\text{lat})$ korekcí; finální výsledek haversine v metrech
- Early stop: když best distance < hranice prstence

5.4 Klíče indexu

```
`` pairKey = SUPER_Z_ID + "|" + SUPER_D_ID roKey = pairKey + "|" + RO_ID ``
```

5.5 GpsMatch

Výsledek findNearest*():

```
``java class GpsMatch { String superZId, superDId, tudu, vyhybka, lat, lon, distanceM, poloha, roId; } ``
```

5.6 SQL fallbacky

Mimo proximity index (např. plný seznam TUDU pro picker):

- `loadAllTudu()`, `loadTuduForUdu()`, `loadTuduForCodes()`
- `findRoBranchesForVyhybka()` - doplní RO větve, pokud index nemá dual-RO
- `queryRoBranchesForVyhybka()` - přímý SQL dotaz

Při prvním GPS lookupu se na kopii DB vytvoří pomocné indexy `_dzs_gps_zd`, `_dzs_gps_ro`.

5.7 Disková cache .pidx (verze 21)

Soubor: `dzs_<hash>_p_<latCell>_<lonCell>.pidx` (GZIP) `latCell = round(lat * 100)`, totéž `lon`.

Binární formát (`DataStream`, big-endian):

```
`` int32 MAGIC = 0x445A5349 ("DZSI") int32 PROXIMITY_VERSION = 21 int64
dbFileLength UTF contentHash (SHA-256 hex, 64 znaků) double centerLat,
centerLon int32 roCount x roCount: UTF pairKey, tudu, int vyhybka, UTF
iob, UTF roId int castMin, castMax, UTF poloha, UTF kmExtChip1, UTF
kmExtOther int32 gpsCount x gpsCount: UTF pairKey, tudu, int vyhybka,
float lat, float lon, UTF roId, UTF poloha ``
```

Platnost:

- hash obsahu DB (async SHA-256, sidecar `hash2_<size>_<mtime>_<pathHash>.txt`)
- vzdálenost středu cache od požadované GPS ≤ 3000 m
- verze 21 (starší `.pidx` / `.idx` se ignorují)

Rychlý klíč: `fastDbKey() = f_<size>_<mtime>` - použít před dokončením async hashe.

Úklid: `deleteObsoleteFullIndexCaches()` maže legacy `dzs_*.idx` (plná indexace).

5.8 VyhybkaGpsStore

Kompaktní columnar storage - pole `pairKeys[]`, `tudus[]`, `vyhybky[]`, `roIds[]`, `polohas[]`, `lats[]` (float), `lons[]` (float). Builder pattern, `appendAll()` pro merge.

6. CSV vrstva

6.1 Formát souboru

Cesta: `Environment.getExternalStoragePublicDirectory(DIRECTORY_DOWNLOADS)/rfid_go_gps_output.csv`

Hlavička (aktuální):

```
` ID_RFID;EPC;TID;TUDU;OBJEKT;POZICE;POLOHA;RO_ID_1;RO_ID_2;KM_EXT;LAT;LON;ACCURACY_M;GPS DATE `
```

Klíč: ID_RFID – upsert přepíše existující řádek.

6.2 CsvStore

Metoda	Účel
<code>upsert(Row) / upsertAndPersist(Row)</code>	vložení + zápis na disk
<code>reloadIfChanged()</code>	SHA-256 porovnání → reload (USB upload z PC)
<code>getMaxIdRfid()</code>	max ID pro obnovu stavu
<code>findRowByTag / findAllRowsByTag</code>	Kontrola (EPC nebo TID)
<code>findRowForCast(tudu, vyhybka, cast)</code>	konkrétní čip
<code>getWrittenCasts(tudu, vyhybka[, roId])</code>	dokončené čipy
<code>removeLast()</code>	smazání posledního záznamu

Index čipů: `castsByVyhybkaRo` – klíč `TUDU\0vyhybka\0roId` → `Set<Integer>` castů.

Legacy formáty (auto-detekce z hlavičky):

- LEGACY_WITH_ROK – sloupec rok
- LEGACY_NO_POLOHA
- LEGACY_NO_RO_KM
- LEGACY_SINGLE_RO – jeden RO_ID sloupec
- CURRENT_KM_EXT_SPLIT – aktuální RO_ID_1/RO_ID_2

6.3 CsvStorage (Android 10+)

Metoda	Chování
<code>resolveFile(Context)</code>	fyzická cesta Download/
<code>openOutputStream()</code>	MediaStore IS_PENDING=1 pro nový soubor
<code>publishForMtp()</code>	IS_PENDING=0 + media scan

Pokud soubor existuje a je zapisovatelný (typicky nahraný z PC přes MTP), používá se přímý `FileOutputStream`.

6.4 CsvRecordBuilder

Čistá factory – odděluje sloupce CSV od layoutu EPC:

```
``java static CsvStore.Row build( long idRfid, String epc24, String tid,  
String tudu, String vyhybkaLabel, int cast, String poloha, String roId1,  
String roId2, String kmExt, String latitude, String longitude, String  
accuracyM, String gpsTime) ``
```

7. UHF / RFID (UhfManager)

Obal nad `com.rscja.deviceapi.RFIDWithUHFUART`.

7.1 Gen2 banky

Konstanta	Hodnota	Obsah
BANK_RESERVED	0	kill/access password
BANK_EPC	1	EPC
BANK_TID	2	TID
BANK_USER	3	user memory

7.2 Zápis EPC

- ptr = 2 (přeskočí CRC+PC), len = 6 wordů = 24 hex
- `writeEpcFromTid(): inventory → read TID → normalizeTidToEpc() → write`
- Normalizace TID: strip non-hex, truncate/pad na 24 znaků trailing 0

7.3 Access heslo a lock

- Heslo: BANK_RESERVED, ptr 2, len 2 (8 hex)
- Lock: `reader.lockMem(accessPwd, lockCode)` – výchozí 008020
- Výchozí nové heslo v UI: 12345678 (NEW_PWD_PRESET)

7.4 Preset fallback

```
``java PRESET_ACCESS_PASSWORDS = {"11223344", "11112222", "12345678"} ``
```

`writeDataWithPresetFallback()`: zkusí uživatelské heslo → iterace presetů.
`WriteResult.usedPresetPassword / presetPasswordUsed` – UI po úspěchu resetuje access pole na preset.

7.5 Výkon

`setPower(int dbm)` – presety v MainActivity:

- V koleji: 16 dBm (`POWER_PRESET_KOLEJI_DBM`)
- V ruce: 1 dBm (`POWER_PRESET_RUCE_DBM`)

8. GPS (LocationCache)

Konstanta	Hodnota
GPS_UPDATE_INTERVAL_MS	500
NETWORK_UPDATE_INTERVAL_MS	2000
STALE_AFTER_MS	30 000
RECENT_FIX_MS	15 000

Výběr fixu: GPS provider preferován před síťovou polohou; při shodě lepší accuracy (± 3 m threshold), pak novější timestamp.

API: `start()`, `stop()`, `getSnapshot()`, `setTestOverride()`, `isStale()`, `formatStatusText()`.

Při zápisu tagu se bere nejlepší známá poloha bez čekání na nový fix.

9. KmExtResolver

```
``java static Values fromOdDoKmRef(Double od, Double doVal, Double kmRef)
``
```

- chip1 = KM_REF formátovaný (formatKm)
- other = druhý konec (OD nebo DO); pokud KM_REF neodpovídá ani jednomu, vybere vzdálenější
- KM_EPS = $1e-4$ pro float porovnání

10. UI a layouty

Soubor	Obsah
activity_main.xml	hlavní obrazovka, overlay Kontrola, indikátory
bottom_sheet_workflow.xml	panel Pokročilé (CSV, EPC šablona, ruční kroky)
dialog_tudu_picker.xml	výběr UDU/TUDU
dialog_tudu_boundary.xml	formulář hranice TUDU
item_kontrola_field*.xml	buňky režimu Kontrola
row_*.xml	řádky EPC šablony a CSV tabulky

CsvAdapter – RecyclerView.Adapter pro 5 posledních řádků v Pokročilých.

FileProvider – sdílení CSV (`${applicationId}.fileprovider`).

11. Build a nasazení

11.1 Gradle

Parametr	Hodnota
compileSdk / targetSdk	34
minSdk	21
Java	17
AGP	8.5.2
Gradle	8.7
ABI	armeabi-v7a, arm64-v8a

Verze: `version.properties` → `APP_VERSION_NAME`, `APP_VERSION_CODE`. APK: `rfid_go_gps_<versionName>.apk`.

11.2 Podpis

`keystore.properties` + `app/keystore/rfid_go_gps_upload.jks` - debug i release stejný klíč (in-place update).

11.3 CI

`.github/workflows/android.yml` - na merge do main: bump verze, `assembleRelease`, GitHub Release s APK.

11.4 Vývoj v Cursor Cloud VM

Viz `AGENTS.md`: JDK 17, SDK `/home/ubuntu/android-sdk`, `local.properties` s `sdk.dir`. APK nelze spustit na x86 emulátoru - ověření buildem + JVM testy čisté logiky (`EpcModel`, ...).

12. Mapa klíčových metod MainActivity

Pro navigaci v ~5 600 řádcích:

Oblast	Metody
Lifecycle	onCreate, onResume, onPause, onDestroy
DB	loadDatabaseFromPath, tryAutoDiscoverDatabase, beginCard1DbLoad
GPS	scheduleGpsTuduLookup, applyGpsMatch, refreshGpsAtWorkflowStart
Pickers	showTuduPicker, showNearbyTuduPicker, showVyhybkaPicker
Zápis	runTriggerAction, doWrite, doWritePassword, doLock
Callbacks	onWriteDone, onPwdWriteDone, onLockDone, onTagCycleComplete
CSV	saveRowToCsv, buildCsvRow, reloadCsvIfChanged
Kontrola	showKontrolaOverlay, runKontrolaRead, showKontrolaMatch
Hranice	showTuduBoundaryForm, applyTuduBoundaryForm, finishTuduBoundaryWriteCycle
UI stav	updateStep1, updateWorkflowIndicators, setActionStatusReady
Trigger	onKeyDown

13. Známá omezení a mrtvý kód

Položka	Stav
UhfManager.readSingle() mimo Kontrolu	nepoužito v hlavním workflow
Hromadné smazání CSV	jen removeLast()
Samostatná Settings activity	neexistuje
Import cizího CSV	ne – jen vlastní výstup
Export XLSX	neimplementováno
Unit/instrumentation testy	složky chybí
docs/INDEXACE_DZS.md	popis plné .idx indexace zastaralý (od v3.141 jen .pidx okolí)

14. Doporučené body pro údržbu

- Unit testy – `EpcModel.buildEpc/decode`, `KmExtResolver`, `CsvStore` legacy migrace, `Tudu`. Vyhybka 4částová logika.
- Refaktor `MainActivity` – extrakce `WorkflowController`, `GpsSelectionController`, `KontrolaController` (bez změny chování).
- Aktualizace `INDEXACE_DZS.md` – sjednotit s modelem `.pidx v21`.
- Perzistentní prefs – `access pwd`, výchozí rok, výkon (viz README „Možné další kroky“).

15. Související dokumenty

Dokument	Účel
docs/prirucka-teren.md	jednoduchá příručka pro terén
docs/prirucka-uzivatele.md	kompletní uživatelská příručka
docs/prirucka-vyvojare.md	tento dokument
docs/INDEXACE_DZS.md	detail indexace DZS (částečně zastaralé)
README.md	přehled projektu a provoz
AGENTS.md	build v Cursor Cloud VM

Generování PDF: `python3 docs/generate_prirucka_vyvojare.py` →
`docs/RFID_Go_GPS_prirucka_vyvojare.pdf`

Technická příručka pro vývojáře – RFID Go GPS verze 3.153. Aktualizujte při změně architektury nebo veřejného API tříd.